

Assignment 1. ETL with Python

Chioma Onyekpere

Predictive Analytics Diploma, PACE, University of Winnipeg

Data Acquisition & Management for Data Science PA F24

Muhammad Shahin

January 19, 2024

lab1

January 19, 2025

1 Objectives

The **objectives** of this assignment are:

1. **Implement an ETL Pipeline:**
 - Design and execute an **Extract, Transform, Load (ETL)** process using Python.
2. **Ingestion and Integration:**
 - Work with data from multiple file formats (CSV, JSON, XML).
 - Combine these data sources into a single, unified dataset using a **Pandas DataFrame**.
3. **Data Transformation:**
 - Perform a simple transformation on the data:
 - Double the values in the **price** column.
 - Round the transformed prices to two decimal points.
4. **Load Transformed Data:**
 - Save the processed and transformed data to a **destination repository** in CSV format.
5. **Log ETL Operations:**
 - Maintain a **log file** that captures timestamps and messages:
 - **[INFO]** for successful steps.
 - **[ERROR]** for any issues encountered.

Step 1: Creating a Suitable Directory Structure To facilitate the **ETL process**, I created a directory structure that separates data and operations based on the ETL phases. This structured approach ensures clarity and easy management of files during each stage of the process. The directories created are as follows:

- **tmp**: A temporary staging area (or landing zone) to store the downloaded ZIP file.
- **data**: A folder to host the extracted data files from the ZIP archive. This serves as the source directory for subsequent operations.
- **output**: A destination repository to store the transformed data as a CSV file.
- **logs**: A folder dedicated to storing the ETL process logs, capturing all key operations and any errors.

This logical organization aligns with best practices in ETL processes, ensuring smooth transitions and clarity between the different stages of the pipeline.

```
[3]: import os
import logging

# Setup logging
```

```

os.makedirs("logs", exist_ok=True)
logging.basicConfig(
    filename='logs/etl.log',
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    filemode='w'  # Ensure the log file is created fresh each run
)

# Create Directory Structure
directories = ["tmp", "data", "output", "logs"]
for directory in directories:
    os.makedirs(directory, exist_ok=True)
logging.info("Directory structure created successfully.")

```

Step 2: Downloading the Zipped Data Folder The next step involved downloading the data from the provided URL:

<https://elasticbeanstalk-us-east-2-340729127361.s3.us-east-2.amazonaws.com/prices.zip>

The ZIP file contains car price datasets in various formats (CSV, JSON, XML). The downloaded file was stored in the **tmp** folder to act as a staging area for further processing. This step is critical for ensuring the availability of raw data needed for the ETL pipeline.

- **Tools Used:**

- Python's **requests** library was used to programmatically download the ZIP file.
- File operations in Python ensured the file was properly saved in the designated folder.

- **Logging:**

- The success of this step was recorded in the **logs** folder using Python's **logging** module.
- Example log entry:

2025-01-16 14:20:00 [INFO] Successfully downloaded the ZIP file to the tmp directory.

By ensuring the data is systematically downloaded and stored in a dedicated folder, I prepared the groundwork for the extraction and transformation phases.

```

[6]: import requests

# Define URL and download path
url = "https://elasticbeanstalk-us-east-2-340729127361.s3.us-east-2.amazonaws.com/prices.zip"
zip_path = "tmp/prices.zip"

# Download the ZIP File
try:
    response = requests.get(url)
    response.raise_for_status()  # Check if the request was successful
    with open(zip_path, "wb") as f:

```

```

        f.write(response.content)
    logging.info("ZIP file downloaded successfully to the tmp directory.")
except requests.exceptions.RequestException as e:
    logging.error(f"Failed to download ZIP file: {e}")
    raise

```

Step 3: Extracting and Organizing the Data Files Once the ZIP file was downloaded, the next step involved extracting its contents and organizing the data files into a dedicated folder. This ensures that the data is easily accessible for integration and transformation steps.

- **Process:**

- The ZIP file located in the **tmp** directory was unzipped using Python's **zipfile** module.
- The extracted files, which include datasets in multiple formats (CSV, JSON, XML), were copied into the **data** folder.

- **Tools Used:**

- Python's **zipfile** module was utilized to extract the contents of the ZIP archive.
- File operations were performed to ensure proper organization of the extracted files.

- **Logging:**

- All steps of the extraction process were logged for transparency and troubleshooting.
- Example log entry:

```
2025-01-16 14:25:00 [INFO] ZIP file extracted successfully to the data directory.
```

```

[9]: import zipfile

# Extract ZIP File
try:
    with zipfile.ZipFile("tmp/prices.zip", 'r') as zip_ref:
        zip_ref.extractall("data")
    logging.info("ZIP file extracted successfully to the data directory.")
except zipfile.BadZipFile as e:
    logging.error(f"Failed to extract ZIP file: {e}")
    raise

```

Step 4: Extracting and Integrating Data into a Common DataFrame The extracted data files in CSV, JSON, and XML formats were loaded and integrated into a single Pandas DataFrame. This unified DataFrame serves as the foundation for subsequent transformation steps.

- **Process:**

- **CSV Files:** Loaded using Pandas' `read_csv()` function.
- **JSON Files:** Parsed using `read_json()` with the `lines=True` parameter for newline-delimited JSON records.
- **XML Files:** Parsed using Python's `xml.etree.ElementTree` module and converted into a DataFrame.

- All DataFrames were concatenated into a single DataFrame using Pandas' `concat()` function.

- **Tools Used:**

- Pandas for data loading and integration.
- Python's standard library for XML parsing.

- **Logging:**

- Logs were created for each successfully processed file format.
- Example log entry:

2025-01-16 14:30:00 [INFO] Successfully loaded and integrated data from CSV, JSON, and

```
[12]: import pandas as pd
import xml.etree.ElementTree as ET
import os

# Function to process files
def process_file(file_path, file_type):
    try:
        if file_type == "csv":
            df = pd.read_csv(file_path)
        elif file_type == "json":
            df = pd.read_json(file_path, lines=True)
        elif file_type == "xml":
            tree = ET.parse(file_path)
            root = tree.getroot()
            rows = [{child.tag: child.text for child in row} for row in root.
↪findall("row")]
            df = pd.DataFrame(rows)
        else:
            raise ValueError(f"Unsupported file type: {file_type}")

        logging.info(f"Successfully processed {file_type.upper()} file: ↵
↪{file_path}")
        return df
    except Exception as e:
        logging.error(f"Error processing {file_type.upper()} file {file_path}: ↵
↪{e}")
        return pd.DataFrame() # Return an empty DataFrame in case of failure
```

```
[14]: def print_dataframe(df, title="DataFrame"):
    """
    Prints a DataFrame with all rows and columns displayed.

    Args:
        df (pd.DataFrame): The DataFrame to print.
```

```

        title (str): Title for the DataFrame display.
    """
    with pd.option_context('display.max_rows', None, 'display.max_columns',
        ↪None):
        print(f"{title}:")
        print(df)

```

```

[16]: # Initialize an empty list to store DataFrames
dataframes = []

# Process all files in the 'data' folder
for file in os.listdir("data"):
    file_path = os.path.join("data", file)
    if file.endswith(".csv"):
        df = process_file(file_path, "csv")
    elif file.endswith(".json"):
        df = process_file(file_path, "json")
    elif file.endswith(".xml"):
        df = process_file(file_path, "xml")
    else:
        logging.warning(f"Skipping unsupported file: {file}")
        continue

    if not df.empty:
        dataframes.append(df)
        print_dataframe(df, title=f"Contents of {file}:")

```

Contents of car_prices1.json::

	car_model	year_of_manufacture	price	fuel
0	ritz	2012	4626.865672	Diesel
1	ritz	2011	3507.462687	Petrol
2	swift	2014	7388.059701	Diesel
3	ertiga	2014	8955.223881	Diesel
4	dzire	2014	8208.955224	Diesel
5	sx4	2011	4402.985075	CNG
6	dzire	2015	6940.298507	Petrol
7	800	2003	522.388060	Petrol
8	alto k10	2016	4477.611940	Petrol
9	sx4	2003	3358.208955	Petrol

Contents of car_prices2.xml::

	car_model	year_of_manufacture	price	fuel
0	etios livia	2014	5895.522388059701	Diesel
1	corolla altis	2011	6716.417910447762	Diesel
2	corolla altis	2013	11119.402985074626	Petrol
3	etios livia	2011	3955.223880597015	Petrol
4	etios cross	2014	7313.432835820897	Diesel
5	etios g	2015	5895.522388059701	Petrol
6	corolla altis	2013	8208.955223880597	Petrol

7	corolla	2004	2238.805970149254	Petrol
8	corolla altis	2010	7835.820895522388	Petrol
9	fortuner	2012	21641.79104477612	Diesel

Contents of car_prices3.xml::

	car_model	year_of_manufacture	price	fuel
0	corolla altis	2016	21985.074626865673	Diesel
1	etios gd	2015	7089.55223880597	Diesel
2	innova	2017	34328.35820895522	Diesel
3	innova	2015	18656.716417910447	Diesel
4	innova	2005	5208.955223880597	Diesel
5	camry	2006	3731.3432835820895	Petrol
6	land cruiser	2010	52238.80597014925	Diesel
7	corolla altis	2012	8805.970149253732	Petrol
8	etios liva	2013	5149.253731343284	Petrol
9	etios g	2014	7089.55223880597	Petrol

Contents of car_prices1.xml::

	car_model	year_of_manufacture	price	fuel
0	corolla altis	2013	10373.134328358208	Petrol
1	etios cross	2015	6716.417910447762	Petrol
2	fortuner	2014	27985.074626865673	Diesel
3	fortuner	2015	35074.62686567164	Diesel
4	fortuner	2017	49253.73134328358	Diesel
5	etios liva	2014	7089.55223880597	Diesel
6	innova	2017	29477.611940298506	Petrol
7	fortuner	2010	13805.970149253732	Diesel
8	corolla altis	2011	6492.537313432835	Petrol
9	corolla altis	2016	21268.65671641791	Petrol

Contents of car_prices1.csv::

	car_model	year_of_manufacture	price	fuel
0	ritz	2014	5000.000000	Petrol
1	sx4	2013	7089.552239	Diesel
2	ciaz	2017	10820.895522	Petrol
3	wagon r	2011	4253.731343	Petrol
4	swift	2014	6865.671642	Diesel
5	vitara brezza	2018	13805.970149	Diesel
6	ciaz	2015	10074.626866	Petrol
7	s cross	2015	9701.492537	Diesel
8	ciaz	2016	13059.701493	Diesel
9	ciaz	2015	11119.402985	Diesel

Contents of car_prices3.csv::

	car_model	year_of_manufacture	price	fuel
0	alto k10	2016	4253.731343	Petrol
1	ignis	2017	7313.432836	Petrol
2	sx4	2011	6567.164179	Petrol
3	alto k10	2014	3731.343284	Petrol
4	wagon r	2013	4328.358209	Petrol
5	swift	2011	4477.611940	Petrol
6	swift	2013	6194.029851	Petrol

7	swift	2017	8955.223881	Petrol
8	alto k10	2010	2910.447761	Petrol
9	ciaz	2015	11119.402985	Diesel

Contents of car_prices2.csv::

	car_model	year_of_manufacture	price	fuel
0	alto 800	2017	4253.731343	Petrol
1	ciaz	2015	10223.880597	Diesel
2	ciaz	2015	11194.029851	Petrol
3	ertiga	2015	9104.477612	Petrol
4	dzire	2009	3358.208955	Petrol
5	ertiga	2016	11567.164179	Diesel
6	ertiga	2015	10820.895522	Diesel
7	ertiga	2016	11567.164179	Diesel
8	wagon r	2015	4850.746269	CNG
9	sx4	2010	3955.223881	Petrol

Contents of car_prices2.json::

	car_model	year_of_manufacture	price	fuel
0	baleno	2016	8731.343284	Petrol
1	alto k10	2014	3805.970149	Petrol
2	sx4	2008	2910.447761	Petrol
3	dzire	2014	8208.955224	Diesel
4	omni	2012	1865.671642	Petrol
5	ciaz	2014	11194.029851	Petrol
6	ritz	2013	3955.223881	Petrol
7	wagon r	2006	1567.164179	Petrol
8	ertiga	2015	8656.716418	Petrol
9	ciaz	2017	11567.164179	Petrol

Contents of car_prices3.json::

	car_model	year_of_manufacture	price	fuel
0	fortuner	2012	22238.805970	Diesel
1	fortuner	2015	34328.358209	Diesel
2	innova	2017	26865.671642	Diesel
3	fortuner	2013	23880.597015	Diesel
4	innova	2005	4104.477612	Petrol
5	corolla altis	2009	5373.134328	Petrol
6	etios cross	2015	6716.417910	Petrol
7	corolla altis	2010	7089.552239	Petrol
8	etios g	2014	6119.402985	Petrol
9	fortuner	2014	29835.820896	Diesel

```
[18]: # Combine all DataFrames
combined_df = pd.concat(dataframes, ignore_index=True)
logging.info("Data successfully integrated into a single DataFrame.")

# Print the merged DataFrame
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    print("Merged DataFrame:")
```



```
print(combined_df)
```

Merged DataFrame:

	car_model	year_of_manufacture	price	fuel
0	ritz	2012	4626.865672	Diesel
1	ritz	2011	3507.462687	Petrol
2	swift	2014	7388.059701	Diesel
3	ertiga	2014	8955.223881	Diesel
4	dzire	2014	8208.955224	Diesel
5	sx4	2011	4402.985075	CNG
6	dzire	2015	6940.298507	Petrol
7	800	2003	522.38806	Petrol
8	alto k10	2016	4477.61194	Petrol
9	sx4	2003	3358.208955	Petrol
10	etios liva	2014	5895.522388059701	Diesel
11	corolla altis	2011	6716.417910447762	Diesel
12	corolla altis	2013	11119.402985074626	Petrol
13	etios liva	2011	3955.223880597015	Petrol
14	etios cross	2014	7313.432835820897	Diesel
15	etios g	2015	5895.522388059701	Petrol
16	corolla altis	2013	8208.955223880597	Petrol
17	corolla	2004	2238.805970149254	Petrol
18	corolla altis	2010	7835.820895522388	Petrol
19	fortuner	2012	21641.79104477612	Diesel
20	corolla altis	2016	21985.074626865673	Diesel
21	etios gd	2015	7089.55223880597	Diesel
22	innova	2017	34328.35820895522	Diesel
23	innova	2015	18656.716417910447	Diesel
24	innova	2005	5208.955223880597	Diesel
25	camry	2006	3731.3432835820895	Petrol
26	land cruiser	2010	52238.80597014925	Diesel
27	corolla altis	2012	8805.970149253732	Petrol
28	etios liva	2013	5149.253731343284	Petrol
29	etios g	2014	7089.55223880597	Petrol
30	corolla altis	2013	10373.134328358208	Petrol
31	etios cross	2015	6716.417910447762	Petrol
32	fortuner	2014	27985.074626865673	Diesel
33	fortuner	2015	35074.62686567164	Diesel
34	fortuner	2017	49253.73134328358	Diesel
35	etios liva	2014	7089.55223880597	Diesel
36	innova	2017	29477.611940298506	Petrol
37	fortuner	2010	13805.970149253732	Diesel
38	corolla altis	2011	6492.537313432835	Petrol
39	corolla altis	2016	21268.65671641791	Petrol
40	ritz	2014	5000.0	Petrol
41	sx4	2013	7089.552239	Diesel
42	ciaz	2017	10820.895522	Petrol
43	wagon r	2011	4253.731343	Petrol

44	swift	2014	6865.671642	Diesel
45	vitara brezza	2018	13805.970149	Diesel
46	ciaz	2015	10074.626866	Petrol
47	s cross	2015	9701.492537	Diesel
48	ciaz	2016	13059.701493	Diesel
49	ciaz	2015	11119.402985	Diesel
50	alto k10	2016	4253.731343	Petrol
51	ignis	2017	7313.432836	Petrol
52	sx4	2011	6567.164179	Petrol
53	alto k10	2014	3731.343284	Petrol
54	wagon r	2013	4328.358209	Petrol
55	swift	2011	4477.61194	Petrol
56	swift	2013	6194.029851	Petrol
57	swift	2017	8955.223881	Petrol
58	alto k10	2010	2910.447761	Petrol
59	ciaz	2015	11119.402985	Diesel
60	alto 800	2017	4253.731343	Petrol
61	ciaz	2015	10223.880597	Diesel
62	ciaz	2015	11194.029851	Petrol
63	ertiga	2015	9104.477612	Petrol
64	dzire	2009	3358.208955	Petrol
65	ertiga	2016	11567.164179	Diesel
66	ertiga	2015	10820.895522	Diesel
67	ertiga	2016	11567.164179	Diesel
68	wagon r	2015	4850.746269	CNG
69	sx4	2010	3955.223881	Petrol
70	baleno	2016	8731.343284	Petrol
71	alto k10	2014	3805.970149	Petrol
72	sx4	2008	2910.447761	Petrol
73	dzire	2014	8208.955224	Diesel
74	omni	2012	1865.671642	Petrol
75	ciaz	2014	11194.029851	Petrol
76	ritz	2013	3955.223881	Petrol
77	wagon r	2006	1567.164179	Petrol
78	ertiga	2015	8656.716418	Petrol
79	ciaz	2017	11567.164179	Petrol
80	fortuner	2012	22238.80597	Diesel
81	fortuner	2015	34328.358209	Diesel
82	innova	2017	26865.671642	Diesel
83	fortuner	2013	23880.597015	Diesel
84	innova	2005	4104.477612	Petrol
85	corolla altis	2009	5373.134328	Petrol
86	etios cross	2015	6716.41791	Petrol
87	corolla altis	2010	7089.552239	Petrol
88	etios g	2014	6119.402985	Petrol
89	fortuner	2014	29835.820896	Diesel

Step 5: Transforming Data and Saving to Destination Folder The integrated DataFrame underwent a simple transformation to modify the `price` column, doubling its values and rounding them to two decimal places. The transformed data was then saved in CSV format to the **output** folder.

- **Process:**

- The `price` column was multiplied by 2 and rounded to two decimal points using Pandas operations.
- The transformed DataFrame was saved as a CSV file in the **output** folder.

- **Tools Used:**

- Pandas for data transformation and saving the file.

- **Logging:**

- Logs were created to indicate the completion of data transformation and saving.
- Example log entry:

```
2025-01-16 14:35:00 [INFO] Data transformed and saved to output/transformed_data.csv.
```

```
[21]: # Transform Data
combined_df['price'] = combined_df['price'].astype(float) * 2
combined_df['price'] = combined_df['price'].round(2)
logging.info("Data transformed successfully.")

# Save Transformed Data
output_path = "output/transformed_data.csv"
combined_df.to_csv(output_path, index=False)
logging.info(f"Transformed data saved to {output_path}.")

# Print the transformed DataFrame
print_dataframe(combined_df, title="Transformed DataFrame:")
```

Transformed DataFrame::

	car_model	year_of_manufacture	price	fuel
0	ritz	2012	9253.73	Diesel
1	ritz	2011	7014.93	Petrol
2	swift	2014	14776.12	Diesel
3	ertiga	2014	17910.45	Diesel
4	dzire	2014	16417.91	Diesel
5	sx4	2011	8805.97	CNG
6	dzire	2015	13880.60	Petrol
7	800	2003	1044.78	Petrol
8	alto k10	2016	8955.22	Petrol
9	sx4	2003	6716.42	Petrol
10	etios liva	2014	11791.04	Diesel
11	corolla altis	2011	13432.84	Diesel
12	corolla altis	2013	22238.81	Petrol
13	etios liva	2011	7910.45	Petrol

14	etios cross	2014	14626.87	Diesel
15	etios g	2015	11791.04	Petrol
16	corolla altis	2013	16417.91	Petrol
17	corolla	2004	4477.61	Petrol
18	corolla altis	2010	15671.64	Petrol
19	fortuner	2012	43283.58	Diesel
20	corolla altis	2016	43970.15	Diesel
21	etios gd	2015	14179.10	Diesel
22	innova	2017	68656.72	Diesel
23	innova	2015	37313.43	Diesel
24	innova	2005	10417.91	Diesel
25	camry	2006	7462.69	Petrol
26	land cruiser	2010	104477.61	Diesel
27	corolla altis	2012	17611.94	Petrol
28	etios liva	2013	10298.51	Petrol
29	etios g	2014	14179.10	Petrol
30	corolla altis	2013	20746.27	Petrol
31	etios cross	2015	13432.84	Petrol
32	fortuner	2014	55970.15	Diesel
33	fortuner	2015	70149.25	Diesel
34	fortuner	2017	98507.46	Diesel
35	etios liva	2014	14179.10	Diesel
36	innova	2017	58955.22	Petrol
37	fortuner	2010	27611.94	Diesel
38	corolla altis	2011	12985.07	Petrol
39	corolla altis	2016	42537.31	Petrol
40	ritz	2014	10000.00	Petrol
41	sx4	2013	14179.10	Diesel
42	ciaz	2017	21641.79	Petrol
43	wagon r	2011	8507.46	Petrol
44	swift	2014	13731.34	Diesel
45	vitara brezza	2018	27611.94	Diesel
46	ciaz	2015	20149.25	Petrol
47	s cross	2015	19402.99	Diesel
48	ciaz	2016	26119.40	Diesel
49	ciaz	2015	22238.81	Diesel
50	alto k10	2016	8507.46	Petrol
51	ignis	2017	14626.87	Petrol
52	sx4	2011	13134.33	Petrol
53	alto k10	2014	7462.69	Petrol
54	wagon r	2013	8656.72	Petrol
55	swift	2011	8955.22	Petrol
56	swift	2013	12388.06	Petrol
57	swift	2017	17910.45	Petrol
58	alto k10	2010	5820.90	Petrol
59	ciaz	2015	22238.81	Diesel
60	alto 800	2017	8507.46	Petrol
61	ciaz	2015	20447.76	Diesel

62	ciaz	2015	22388.06	Petrol
63	ertiga	2015	18208.96	Petrol
64	dzire	2009	6716.42	Petrol
65	ertiga	2016	23134.33	Diesel
66	ertiga	2015	21641.79	Diesel
67	ertiga	2016	23134.33	Diesel
68	wagon r	2015	9701.49	CNG
69	sx4	2010	7910.45	Petrol
70	baleno	2016	17462.69	Petrol
71	alto k10	2014	7611.94	Petrol
72	sx4	2008	5820.90	Petrol
73	dzire	2014	16417.91	Diesel
74	omni	2012	3731.34	Petrol
75	ciaz	2014	22388.06	Petrol
76	ritz	2013	7910.45	Petrol
77	wagon r	2006	3134.33	Petrol
78	ertiga	2015	17313.43	Petrol
79	ciaz	2017	23134.33	Petrol
80	fortuner	2012	44477.61	Diesel
81	fortuner	2015	68656.72	Diesel
82	innova	2017	53731.34	Diesel
83	fortuner	2013	47761.19	Diesel
84	innova	2005	8208.96	Petrol
85	corolla altis	2009	10746.27	Petrol
86	etios cross	2015	13432.84	Petrol
87	corolla altis	2010	14179.10	Petrol
88	etios g	2014	12238.81	Petrol
89	fortuner	2014	59671.64	Diesel

```
[23]: # Read and print the saved CSV to verify contents
transformed_df = pd.read_csv(output_path)
transformed_df
```

```
[23]:
```

	car_model	year_of_manufacture	price	fuel
0	ritz	2012	9253.73	Diesel
1	ritz	2011	7014.93	Petrol
2	swift	2014	14776.12	Diesel
3	ertiga	2014	17910.45	Diesel
4	dzire	2014	16417.91	Diesel
..	...	...	...	...
85	corolla altis	2009	10746.27	Petrol
86	etios cross	2015	13432.84	Petrol
87	corolla altis	2010	14179.10	Petrol
88	etios g	2014	12238.81	Petrol
89	fortuner	2014	59671.64	Diesel

[90 rows x 4 columns]

Summary and Concluding Remarks The ETL pipeline successfully ingested, transformed, and saved the data in a well-structured format. Here are the key accomplishments:

- **Directory Structure:** Organized directories ensured clarity and seamless data processing.
- **Data Ingestion:** Data from CSV, JSON, and XML formats was accurately loaded and integrated.
- **Data Transformation:** Prices were doubled and rounded, meeting the transformation criteria.
- **Logging:** Comprehensive logs captured the entire ETL process, ensuring traceability and easy debugging.

1.0.1 Observations and Lessons Learned

- The variety of file formats provided an opportunity to implement robust file handling techniques.
- Parsing XML required additional effort compared to CSV and JSON but was successfully managed using Python's `ElementTree` module.
- Proper logging greatly simplified tracking progress and diagnosing issues during implementation.

Overall, this project demonstrated the importance of structuring ETL processes for scalability, reliability, and clarity.

References

AWS Elastic Beanstalk. (n.d.). Prices data set [ZIP file]. <https://elasticbeanstalk-us-east-2-340729127361.s3.us-east-2.amazonaws.com/prices.zip>